

Understanding complete flow of design from RTL to GDSII using OpenLane

INTRODUCTION:

The OpenLane full flow provides a complete automated RTL-to-GDSII digital design, it synthesis with Yosys, floorplanning and placement via OpenROAD tools, clock tree synthesis, global and detailed routing, static timing analysis with OpenSTA, power analysis, DRC/LVS verification with Magic and Netgen, and final GDSII streaming all configurable through simple TCL parameters for core utilization, die size, timing constraints, and PDK selection.

APPLICATION:

The OpenLane Full Flow Lab is primarily used to execute and analyse the complete RTL-to-GDSII pipeline for digital ASIC development. It helps students, researchers, and engineers master end-to-end automation from Verilog source to foundry-ready layout, demonstrating how synthesis, floorplanning, placement, CTS, routing, and signoff interact to achieve timing closure and DRC-clean designs. The lab is widely applicable in academic chip tapeouts, RISC-V processor hardening, open-PDK prototyping, SoC integration, and commercial-grade verification flows. Proper execution identifies PPA bottlenecks early, validates open-source EDA maturity, and prepares designs for advanced nodes while minimizing manual intervention across the physical design hierarchy.

OBJECTIVE:

After completing this lab, you will be able to:

- Understand the RTL-to-GDSII flow
- Configure core area, utilization, and aspect ratio parameters in OpenLane
- Place I/O pins, power rings, and standard cell rows effectively.
- Perform global and detailed placement using OpenLane.
- Generate a balanced clock tree using OpenLane.
- Analyse routing congestion and metal layer utilization
- Generating GDSII file

PROCEDURE TO CREATE A LAB:

Step 1: Set up OpenLane flow

1.1: Invoke OpenLane in the terminal using the command: openlane

```
coe-4@ip-10-100-19-29:~/OpenLaneUser/designs$ openlane
=====
OpenLane started for user: coe-4
Workspace : /home/coe-4/OpenLaneUser
PDK Root  : /labroot/openlane_pdk
=====
OpenLane Container (ff5509f):/openlane$
```

1.2: Open folder- designs: cd designs

Step 2: Set up Project Directory

2.1: Create a directory for your lab: mkdir <project_name>

2.2: Open lab directory: cd <project_name>

2.3: Create directory for project: mkdir fd

2.4: Open project directory: cd fd

2.5: Inside project directory, create folder: mkdir src

2.6: Open folder: cd src

Step 3: Create Design file

3.1: Open vi editor for design with command: vi fd.v

RTL Design Code:

```
module fd(
    input clk,
    input reset,
    output clk_out);
    reg[1:0] pos_cnt;
    reg [1:0] neg_cnt;

    always@(posedge clk)
        if(reset)
            pos_cnt<= 0;
        else if(pos_cnt==2)
```

```
        pos_cnt<= 0;

    else

        pos_cnt<= pos_cnt+1;

    always@(negedge clk)
    if(reset)
        neg_cnt<= 0;
    else if(neg_cnt==2)
        neg_cnt<= 0;
    else neg_cnt<= neg_cnt+1;
    assign clk_out=((pos_cnt==2) | (neg_cnt==2));

endmodule
```

3.2: save the file using command- :wq

Note: After writing your RTL design, exit from current src directory and make config.json inside fd directory

Step 4: Create .json file

4.1: Open vi editor: vi config.json

```
{

    "DESIGN_NAME": "fd",
    "VERILOG_FILES": "dir::src/*.v",
    "RUN_CTS": false,
    "CLOCK_PORT": null,
    "PL_RANDOM_GLB_PLACEMENT": true,
    "FP_SIZING": "absolute",
    "DIE_AREA": "0 0 34.5 57.12",
    "PL_TARGET_DENSITY": 0.75,
    "FP_PDN_AUTO_ADJUST": false,
    "FP_PDN_VPITCH": 25,
    "FP_PDN_HPITCH": 25,
    "FP_PDN_VOFFSET": 5,
    "FP_PDN_HOFFSET": 5,
```

```
"DIODE_INSERTION_STRATEGY": 3  
}
```

Step 5: Use the following commands to run the flow:

OpenLane to start the design flow in interactive mode

i) `flow.tcl -interactive`

```
OpenLane Container (ff5509f):/openlane/designs/fd$ flow.tcl -interactive  
OpenLane v1.0.2 (ff5509f65b17bfa4068d5336495ab1718987ff69)  
All rights reserved. (c) 2020-2025 Efabless Corporation and contributors.  
Available under the Apache License, version 2.0. See the LICENSE file for more details.
```

Used in the OpenLane interactive shell to prepare your design for the digital IC design flow.

ii) `prep -design fd`

```
% prep -design fd  
[INFO]: Using configuration in 'config.json'...  
[INFO]: PDK Root: /openlane/pdks  
[INFO]: Process Design Kit: skyl130A  
[INFO]: Standard Cell Library: skyl130_fd_sc_hd  
[INFO]: Optimization Standard Cell Library: skyl130_fd_sc_hd  
[INFO]: Run Directory: /openlane/designs/fd/runs/RUN_2026.01.27_09.38.56  
[INFO]: Saving runtime environment...  
[INFO]: Preparing LEF files for the nom corner...  
[INFO]: Preparing LEF files for the min corner...  
[INFO]: Preparing LEF files for the max corner...
```

Runs yosys synthesis on the current design.

iii) `run_synthesis`

```
% run_synthesis  
[STEP 1]  
[INFO]: Running Synthesis (log: ../runs/RUN_2026.01.27_12.01.37/logs/synthesis/1-synthesis.log)...  
[STEP 2]  
[INFO]: Running Single-Corner Static Timing Analysis (log: ../runs/RUN_2026.01.27_12.01.37/logs/synthesis/2-sta.log)...
```

To initiate floorplan

iv) `run_floorplan`

```
% run_floorplan  
[STEP 3]  
[INFO]: Running Initial Floorplanning (log: ../runs/RUN_2026.01.27_12.01.37/logs/floorplan/3-initial_fp.log)...  
[INFO]: Floorplanned with width 259.44 and height 258.4.  
[STEP 4]  
[INFO]: Running IO Placement...  
[STEP 5]  
[INFO]: Running Tap/Decap Insertion (log: ../runs/RUN_2026.01.27_12.01.37/logs/floorplan/5-tap.log)...  
[INFO]: Power planning with power {VPWR} and ground {VGND}...  
[STEP 6]  
[INFO]: Generating PDN (log: ../runs/RUN_2026.01.27_12.01.37/logs/floorplan/6-pdn.log)...
```

To initiate placement

v) run_placement

```
% run_placement
[STEP 7]
[INFO]: Running Global Placement (log: ../runs/RUN_2026.01.27_12.01.37/logs/placement/7-global.log)...
[STEP 8]
[INFO]: Running Single-Corner Static Timing Analysis (log: ../runs/RUN_2026.01.27_12.01.37/logs/placement/8-gpl_sta.log)...
[STEP 9]
[INFO]: Running Placement Resizer Design Optimizations (log: ../runs/RUN_2026.01.27_12.01.37/logs/placement/9-resizer.log)...
[STEP 10]
[INFO]: Running Detailed Placement (log: ../runs/RUN_2026.01.27_12.01.37/logs/placement/10-detailed.log)...
[STEP 11]
[INFO]: Running Single-Corner Static Timing Analysis (log: ../runs/RUN_2026.01.27_12.01.37/logs/placement/11-dpl_sta.log)...
```

To initiate Clock Tree Synthesis

vi) run_cts

```
% run_cts
[STEP 12]
[INFO]: Running Clock Tree Synthesis (log: ../runs/RUN_2026.01.27_12.01.37/logs/cts/12-cts.log)...
[STEP 13]
[INFO]: Running Single-Corner Static Timing Analysis (log: ../runs/RUN_2026.01.27_12.01.37/logs/cts/13-cts_sta.log)...
```

To initiate routing

vii) run_routing

```
% run_routing
[STEP 14]
[INFO]: Running Global Routing Resizer Design Optimizations (log: ../runs/RUN_2026.01.27_12.01.37/logs/routing/14-resizer_design.log)...
[STEP 15]
[INFO]: Running Single-Corner Static Timing Analysis (log: ../runs/RUN_2026.01.27_12.01.37/logs/routing/15-rsz_design_sta.log)...
[STEP 16]
[INFO]: Running Global Routing Resizer Timing Optimizations (log: ../runs/RUN_2026.01.27_12.01.37/logs/routing/16-resizer_timing.log)...
[STEP 17]
[INFO]: Running Single-Corner Static Timing Analysis (log: ../runs/RUN_2026.01.27_12.01.37/logs/routing/17-rsz_timing_sta.log)...
[STEP 18]
[INFO]: Running Global Routing (log: ../runs/RUN_2026.01.27_12.01.37/logs/routing/18-global.log)...
[INFO]: Starting OpenROAD Antenna Repair Iterations...
[STEP 19]
[INFO]: Writing Verilog (log: ../runs/RUN_2026.01.27_12.01.37/logs/routing/18-global_write_netlist.log)...
[STEP 20]
[INFO]: Running Single-Corner Static Timing Analysis (log: ../runs/RUN_2026.01.27_12.01.37/logs/routing/20-grt_sta.log)...
[STEP 21]
[INFO]: Running Fill Insertion (log: ../runs/RUN_2026.01.27_12.01.37/logs/routing/21-fill.log)...
[STEP 22]
[INFO]: Running Detailed Routing (log: ../runs/RUN_2026.01.27_12.01.37/logs/routing/22-detailed.log)...
[INFO]: No DRC violations after detailed routing.
[STEP 23]
[INFO]: Checking Wire Lengths (log: ../runs/RUN_2026.01.27_12.01.37/logs/routing/23-wire_lengths.log)...
1769515510
```

To initiate magic tool

viii) run_magic

```
% run_magic
[STEP 24]
[INFO]: Running Magic to generate various views...
[INFO]: Streaming out GDSII with Magic (log: ../runs/RUN_2026.01.27_12.01.37/logs/signoff/24-gdsii.log)...
[INFO]: Generating MAGLEF views...
[INFO]: Generating lef with Magic (/openlane/designs/fd/runs/RUN_2026.01.27_12.01.37/logs/signoff/24-lef.log)...
```

For DRC check

ix) run_magic_drc

```
% run_magic_drc
[STEP 25]
[INFO]: Running Magic DRC (log: ../runs/RUN_2026.01.27_12.01.37/logs/signoff/25-drc.log)...
[INFO]: Converting Magic DRC database to various tool-readable formats...
[INFO]: No DRC violations after GDS streaming out.
```

#For Spice extraction

x) run_magic_spice_export

```
% run_magic_spice_export  
[STEP 26]  
[INFO]: Running Magic Spice Export from LEF (log: ../runs/RUN_2026.01.27_12.01.37/logs/signoff/26-spice.log)...
```

For klayout

xi) run_klayout

```
% run_klayout  
[STEP 27]  
[INFO]: Streaming out GDSII with KLayout (log: ../runs/RUN_2026.01.27_12.01.37/logs/signoff/27-gdsii-klayout.log)...
```

#For LVS

xii) run_lvs

```
% run_lvs  
[STEP 28]  
[INFO]: Writing Powered Verilog (logs: ../runs/RUN_2026.01.27_12.01.37/logs/signoff/28-write_powered_def.log, ../runs/RUN_2026.01.27_12.01.37/logs/signoff/28-write_powered_verilog.log)..  
[STEP 29]  
[INFO]: Writing Verilog (log: ../runs/RUN_2026.01.27_12.01.37/logs/signoff/28-write_powered_verilog.log)..  
[STEP 30]  
[INFO]: Running LVS (log: ../runs/RUN_2026.01.27_12.01.37/logs/signoff/30-lvs.lef.log)...
```

```
[INFO]: Saving runtime environment..  
[INFO]: Generating final set of reports..  
[INFO]: Created manufacturability report at 'runs/RUN_2026.01.28_04.05.03/reports/manufacturability.rpt'.  
[INFO]: Created metrics report at 'runs/RUN_2026.01.28_04.05.03/reports/metrics.csv'.  
[INFO]: There are no max slew, max fanout or max capacitance violations in the design at the typical corner.  
[INFO]: There are no hold violations in the design at the typical corner.  
[INFO]: There are no setup violations in the design at the typical corner.  
[SUCCESS]: Flow complete.
```

OBSERVATION:

- Final GDSII layout stream and hierarchical LEF/DEF files for foundry submission
- Timing slack reports from OpenSTA (setup/hold violations, WNS, TNS metrics)
- Power analysis results (dynamic leakage, total power in mW, IR drop maps)
- DRC/LVS clean signoff deck with Magic violation counts and density checks
- Final die utilization vs target, routing congestion heatmaps, and via overflow stats
- Clock tree quality metrics (skew, insertion delay, buffer counts per clock domain)
- Synthesis to final netlist area reduction percentage and cell count breakdown
- Routed wirelength totals, global/detailed routing completion percentages
- Floorplan DEF visualization in KLayout with core/IO/power ring overlays

CONCLUSION:

In this OpenLane full flow lab, the complete RTL-to-GDSII pipeline was successfully executed to produce a production-ready ASIC layout from the given Verilog RTL design. All stages—from synthesis through floorplanning, placement, clock tree synthesis, routing, static timing

analysis, power analysis, and DRC/LVS signoff—completed without critical violations, achieving timing closure with positive slack margins and clean physical verification decks. The generated GDSII, hierarchical DEF/LEF files, and comprehensive reports confirmed optimal die utilization, routability, power integrity, and clock quality metrics.

etchiphub.in